

SWIPE TO LEARN

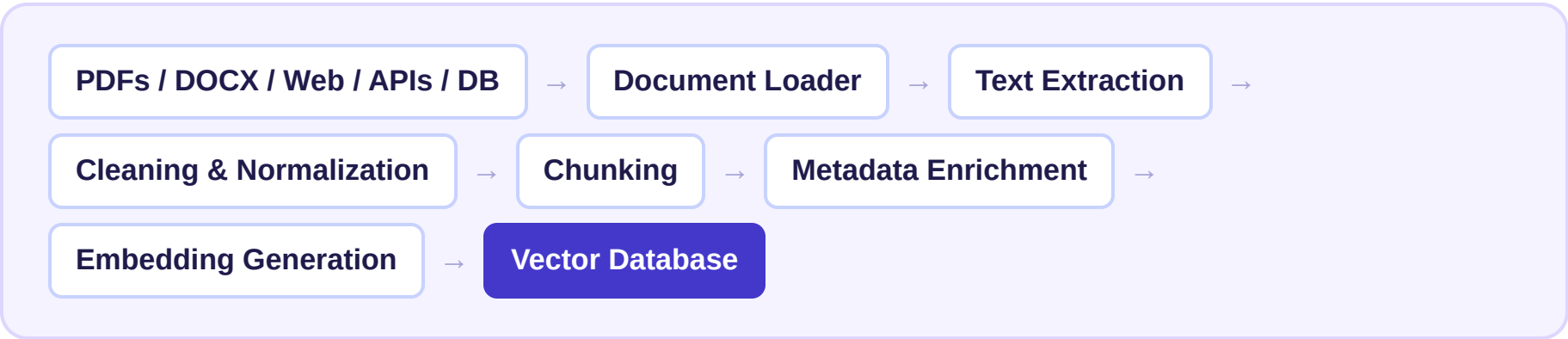
Production-Ready RAG System Architecture

How enterprise RAG systems retrieve relevant knowledge,
ground LLM responses, and reduce hallucinations.



Created By Kartik Mungole

1. Document Ingestion Pipeline



Preserve metadata on every chunk:

- document_id
- title
- source
- page_number
- department
- timestamp
- access_permissions

KEY TAKEAWAY

Bad ingestion usually leads to bad retrieval — even if the LLM is powerful.

2. Fixed-Size vs Semantic Chunking



A document split every ~500 tokens, with a 50-token overlap between chunks

Fixed-Size Chunking

ADVANTAGES

- Simple & fast
- Predictable
- Easy to implement

LIMITATIONS

- Can split sentences
- May lose context

Semantic Chunking

SPLIT BY

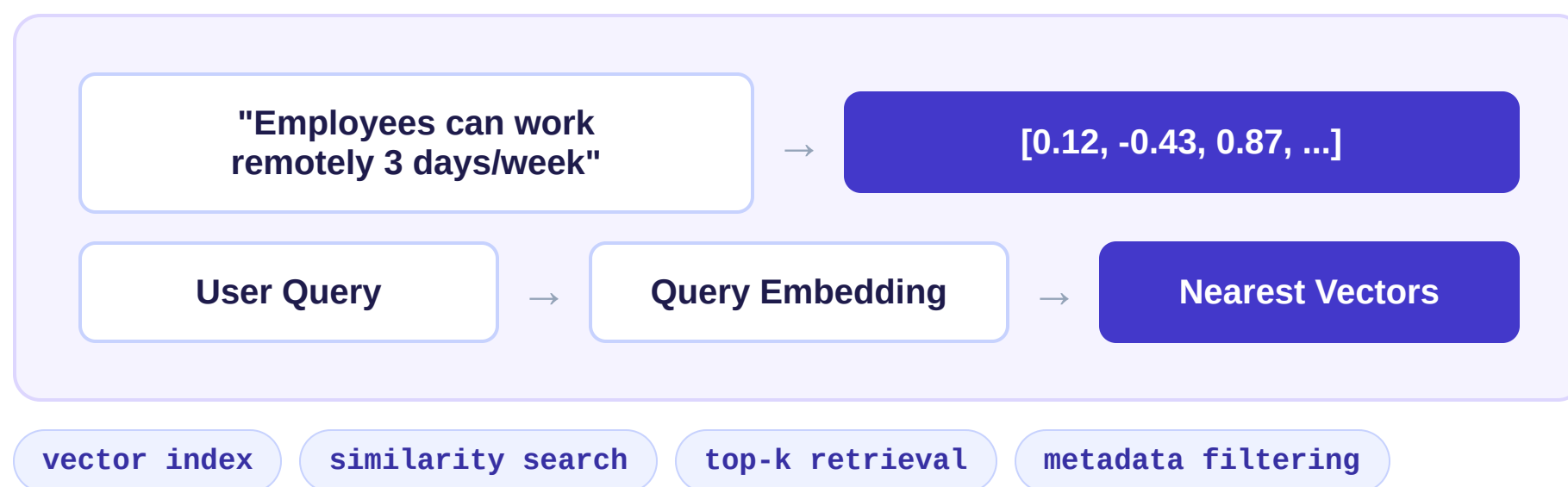
- Paragraphs & sections
- Topic / meaning shifts
- Headings

TRADE - OFF

- More coherent
- More complex & costly

There is no universal "best" chunk size — evaluate it against document type, query type, retrieval quality, and context-window limits.

3. Turning Text into Searchable Knowledge

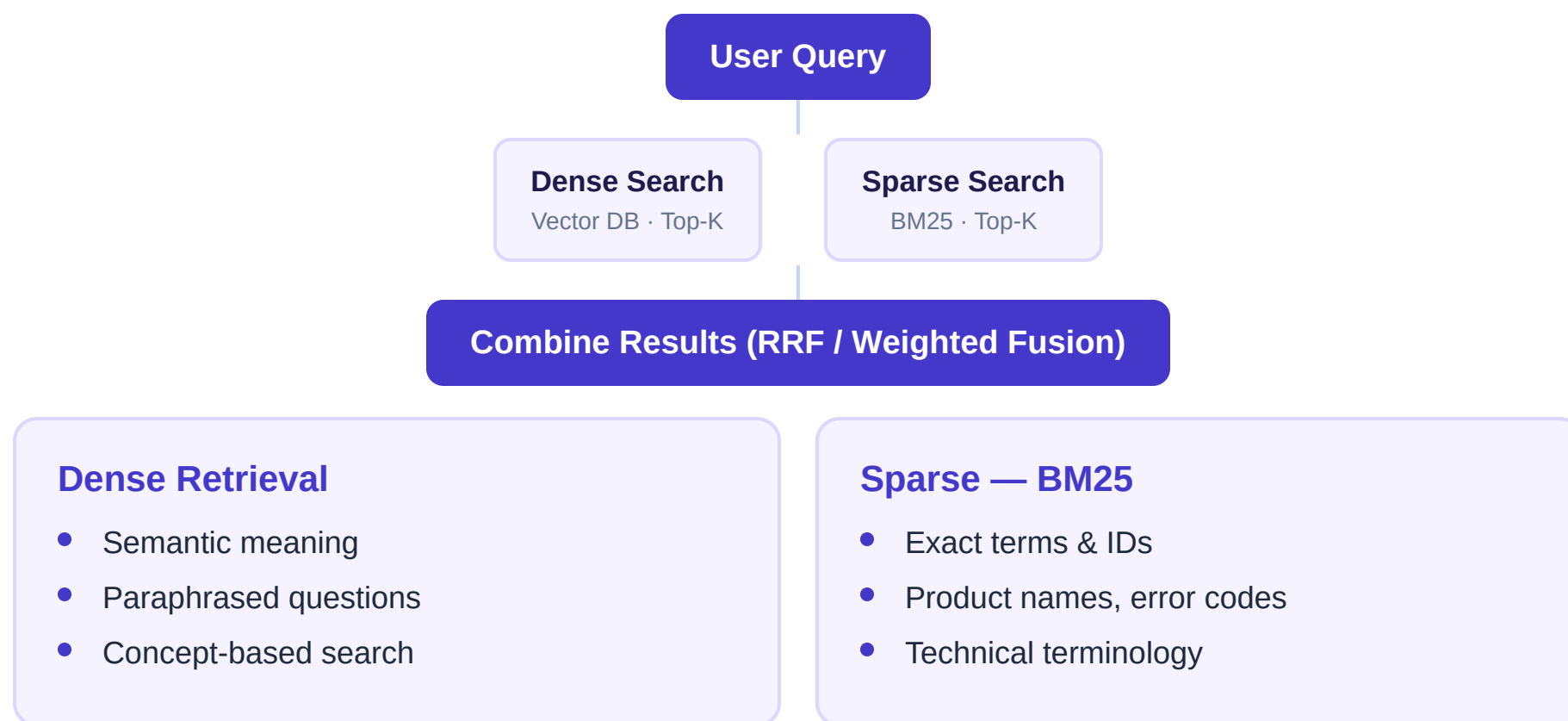


Filters narrow the search space before similarity even runs — e.g. **department = Engineering** or **access_level = employee**.

PRODUCTION POINT

Don't rely only on vector similarity. Metadata and access-control filtering are critical in enterprise RAG.

4. Retrieval: Dense + Sparse Search



5. Reranking: Find the Most Relevant Context



A cross-encoder scores **[Query + Chunk]** together, instead of comparing embeddings independently:

Chunk	Relevance Score
Chunk A	0.94
Chunk B	0.87
Chunk C	0.31

KEY TAKEAWAY

Retrieval optimizes recall. Reranking improves precision.

6. Building the Grounded Prompt

Question + Chunks + Instructions + History

→

Prompt Builder

→

LLM

SYSTEM

You are an enterprise assistant.
Answer using only the provided context.

CONTEXT

[Chunk 1] [Chunk 2] [Chunk 3]

QUESTION

What is the company's remote work policy?

- Separate instructions clearly from retrieved data
- Control total context size & remove duplicate chunks
- Tell the model what to do when the answer is missing

RAG does not magically eliminate hallucinations — better retrieval and grounding instructions reduce unsupported answers.

7. LLM Generation

Grounded Prompt

→

LLM

→

Generated Answer

hosted APIs

self-hosted models

streaming

token limits

temperature

model routing

QUESTION – "What is our leave policy?"

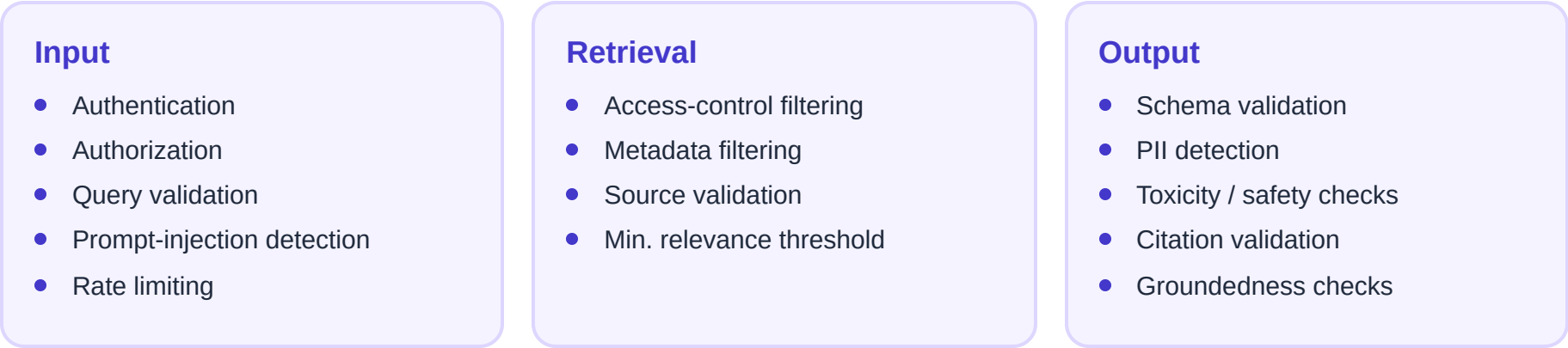
CONTEXT – "Employees receive 24 paid leave days per year."

ANSWER – "Employees receive 24 paid leave days per year."

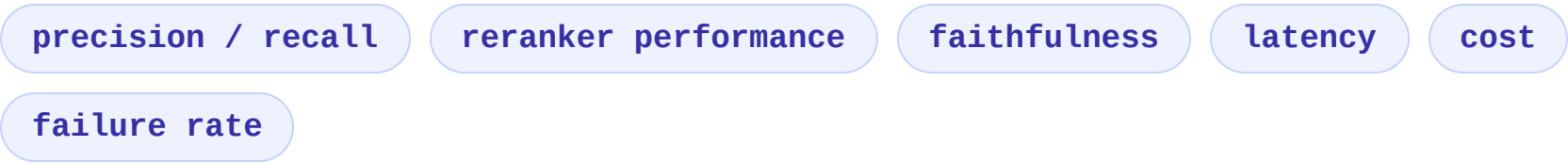
IMPORTANT DISTINCTION

The LLM is not the knowledge database — retrieval supplies external knowledge at query time.

8. Guardrails: RAG Needs More Than an LLM



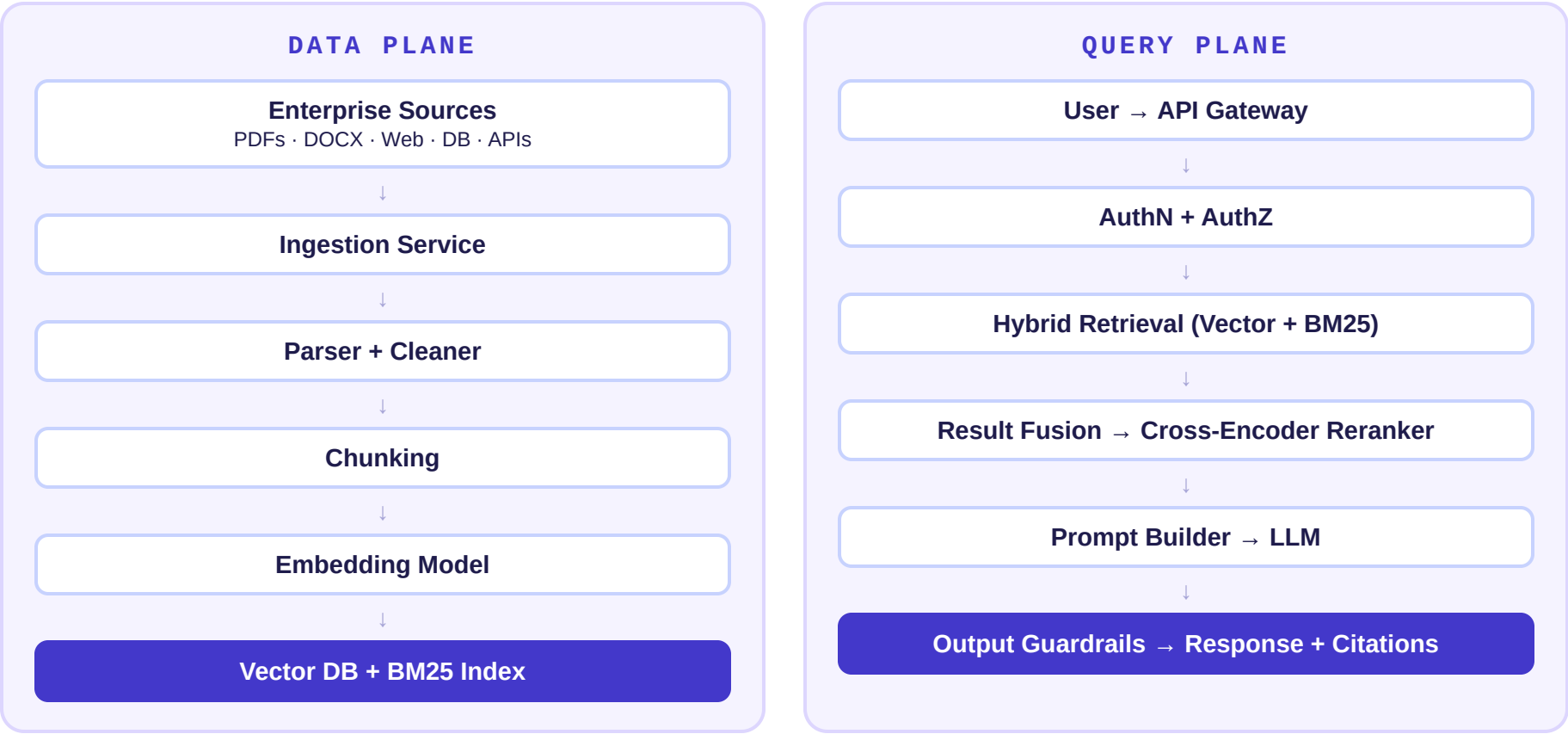
Observability & Evaluation:



ENTERPRISE RULE

A RAG system must respect document-level permissions — a user should never retrieve what they aren't authorized to access.

9. The Complete Production-Ready RAG Architecture



Supporting Infrastructure

Redis / Cache Queue / Workers PostgreSQL Object Storage Monitoring Logging Tracing Evaluation Pipeline

Production RAG = Retrieval + Reranking + Grounded Generation + Security + Observability

- 1. Good RAG starts with good data ingestion.
- 2. Chunking directly affects retrieval quality.
- 3. Hybrid search combines semantic + keyword retrieval.
- 4. Reranking improves retrieved context quality.
- 5. Guardrails & access control are essential.
- 6. Evaluation & observability enable reliability.